



Reporte de Auditoria Integral

EstudioDeco POS v2

Fecha de emision: 13/04/2026
Clasificacion: CONFIDENCIAL -- Solo uso interno
Stack auditado: FastAPI - SQLite - Python 3.x
Modulos revisados: server.py - modules/database.py - sync_sheets.py

Resumen ejecutivo de hallazgos

3

CRITICOS

5

ALTOS

4

MEDIOS

4

BUGS DIA/SEM.

ADVERTENCIA: Se identificaron vulnerabilidades CRITICAS de seguridad.
No desplegar en produccion sin aplicar el Plan de Remediacion -- Bloque A.

Contenido

- | | | |
|----|--|-------------|
| 1. | Fase 1 -- Hallazgos Financieros | F-1 a F-7 |
| 2. | Fase 2 -- Vulnerabilidades Administrativas | S-1 a S-10 |
| 3. | Plan de Remediacion Arquitectonica | Bloques A-D |
| 4. | Analisis de Discrepancias Diario vs. Semanal | Bug #1 a #4 |
| 5. | Tabla resumen de causas raiz | |

Fase 1 -- Hallazgos Financieros

Integridad transaccional - Concurrencia - Reportes - Rendimiento

CRITICO

F-1 -- Sin proteccion transaccional atomica en cerrar_mesa

Archivo: modules/database.py:363-475

La funcion cerrar_mesa ejecuta entre 5 y 15+ sentencias SQL secuenciales (INSERT ventas -> INSERT venta_detalle x N -> UPDATE stock x N -> UPDATE ordenes -> INSERT gastos [comision]) sin ningun bloque try/except/rollback.

Causa raiz: Si la operacion falla a mitad del camino (ej. error al descontar stock del item N), el registro en ventas ya existe pero faltan filas en venta_detalle. El conn.commit() de la linea 454 solo se ejecuta si no hay excepcion; SQLite en modo autocommit implicito puede crear estados parciales que quedan 'huerfanos'. El resumen del dia sumara una venta sin detalle.

Patron faltante:

```
try:
    # todas las operaciones
    conn.commit()
except Exception:
    conn.rollback()
    raise
```

CRITICO

F-2 -- corregir_venta no reversa ni recalcula la comision de tarjeta

Archivo: modules/database.py:1283-1289

Cuando se corrige el metodo de pago de una venta (ej. de Efectivo a Tarjeta), la funcion solo actualiza metodo_pago, monto_efectivo y monto_tarjeta en la tabla ventas. No crea, elimina, ni ajusta el gasto de 'Comision Tarjeta 4%'.

Impacto en cuadre: Si una venta de \$1,000 Efectivo se corrige a Tarjeta, el balance registrara \$1,000 en tarjeta pero no habra gasto de comision (\$40). Al revés: una venta movida de Tarjeta a Efectivo mantendra un gasto fantasma de \$40 que nunca corresponde a un cobro real.

ALTO

F-3 -- Condicion de carrera en descuento de stock

Archivo: modules/database.py:428-429 y 1184-1185

Ambas rutas (mesa y venta directa) usan la misma sentencia sin verificacion previa ni bloqueo. Si dos cajas procesan simultaneamente el ultimo item del stock, ambas UPDATE pasan y el stock queda en negativo. El reporte de inventario queda inconsistente con la realidad fisica.

```
UPDATE productos SET stock_local = stock_local - ? WHERE id=?
```

ALTO

F-4 -- Comision de tarjeta calculada dos veces para pagos mixtos

Archivo: modules/database.py:1187-1195 vs 440-451

En registrar_venta (ventas directas) la comision se aplica si metodo_pago in ('Tarjeta', 'Mixto'). En cerrar_mesa (ventas por mesa) la misma comision se calcula y registra de forma independiente. Cada ruta produce su propio INSERT en gastos, duplicando el cargo si ambas se invocan para la misma operacion.

ALTO

F-5 -- Folio no atomico: condicion de carrera en numeracion

Archivo: modules/database.py:496-500

El folio se genera con un COUNT(*) seguido de un INSERT independiente. Con dos requests concurrentes al mismo milisegundo, ambos obtienen COUNT=5, ambos generan VTA-20260413-0006. El segundo INSERT falla con violacion de unicidad (si existe UNIQUE) o genera folios duplicados silenciosamente (si no existe). Esto causa que los reportes cuenten ventas duplicadas o pierdan registros.

```
def _generar_folio(conn):
    row = conn.execute(
        'SELECT COUNT(*) as c FROM ventas'
        ' WHERE folio LIKE ?', (f'VTA-{hoy}-%',)).fetchone()
    num = (row['c'] if row else 0) + 1
    return f'VTA-{hoy}-{num:04d}'
```

ALTO

F-6 -- Fuga de tiempo: datetime mezcla localtime con comparaciones de fecha Python

Archivo: modules/database.py:755-891

Las ventas se guardan con created_at DEFAULT (datetime('now','localtime')) a nivel de SQLite, pero la comparacion en resumen usa DATE(v.created_at) = ? donde la fecha viene de Python (date.today()). Si el servidor corre en un timezone diferente al de SQLite o hay un cambio de horario de verano, ventas de las ultimas horas del dia pueden caer en la fecha 'equivocada' del resumen.

Adicionalmente, obtener_resumen_semana en la linea 897 calcula la semana usando hoy.weekday() que asume lunes como dia 0. Las ventas del domingo podrian quedar excluidas hasta el cierre de semana dependiendo de la configuracion del negocio.

MEDIO

F-7 -- N+1 queries en obtener_ventas_dia y obtener_ventas_turno

Archivo: modules/database.py:1214-1224 y 1250-1258

Por cada venta del dia se ejecuta una query adicional para obtener sus items. Con 100 ventas en el dia = 101 queries. Esto ralentiza el reporte en horas pico y puede causar timeouts que el usuario interpreta como 'el reporte no cargo', llevando a recargas y potencialmente a estados inconsistentes.

Fase 2 -- Vulnerabilidades Administrativas y de Seguridad

Autenticacion - Autorizacion - Multitenancy - Audit Trail - Rendimiento

CRITICO

S-1 -- Credenciales en texto plano en el codigo fuente

Archivo: server.py:43-46

Las contraseñas están en texto plano dentro del código versionado en git. Cualquier persona con acceso al repositorio (actual o histórico) tiene las credenciales del sistema. No hay hashing, no hay rotación, no hay revocación posible sin redeploy.

```
SYSTEM_USERS = {  
    "estudiodeco": {"password": "19jul", "role": "deco"},  
    "estacion304": {"password": "telefono", "role": "estacion"},  
}
```

CRITICO

S-2 -- Sin autenticación en la mayoría de los endpoints

Archivo: server.py:200-560

El endpoint /api/login existe (línea 193-198) pero su token/sesión nunca se valida en ningún otro endpoint. No existe Depends(get_current_user), middleware de sesión, ni header de autorización verificado en ningún endpoint.

- POST /api/estacion/gasto: cualquiera puede registrar un gasto sin autenticarse
- DELETE /api/ordenes/{oid}: cualquiera puede cancelar cualquier orden
- DELETE /api/catalog/{pid}: cualquiera puede eliminar productos del catálogo
- PUT /api/catalog/{pid}: cualquiera puede modificar precios

CRITICO

S-3 -- Escalada de privilegios por usuario_id controlado por el cliente

Archivo: server.py:67-69, 77-78, 80-86

Los modelos CerrarMesaReq, AbrirMesaReq, GastoReq y CorteReq aceptan usuario_id directamente del body del request. Un usuario puede enviar el usuario_id de un administrador para registrar transacciones bajo su nombre, manipular el historial, o atribuirse permisos de otro rol. El usuario autenticado debe derivarse del token, no del body.

ALTO

S-4 -- Sin aislamiento de datos entre tiendas (Multitenancy)

Archivo: modules/database.py:522-533 y 541-551

No hay filtrado por tienda_id ni por el usuario autenticado. El operador de Estación 304 puede ver los gastos de Estudio Deco y viceversa. Aplica también a listar_nominas.

```
def listar_gastos(limit=200):  
    # Devuelve TODOS los gastos de TODAS las tiendas  
  
def listar_ingresos(limit=200):  
    # Devuelve TODOS los ingresos de TODOS los usuarios
```

ALTO

S-5 -- Manipulación de precios desde el cliente

Archivo: server.py:56-62

AddItemReq acepta precio_unitario: float sin validación contra el catálogo. Cualquier cajero puede agregar un ítem de \$1,000 a precio \$0, o aplicar descuentos arbitrarios sin que el sistema lo rechace o registre como excepción.

ALTO

S-6 -- Hard-delete en gastos e ingresos: sin audit trail*Archivo: modules/database.py:535-538 y 553-556*

Las anulaciones eliminan físicamente los registros. No hay campo anulado, no hay timestamp de quien anulo ni cuando. Esto rompe la trazabilidad contable: si un empleado registra un ingreso de \$5,000, lo 'anula', y registra otro por \$3,000, no hay evidencia de la discrepancia en la base de datos.

```
def anular_gasto(gasto_id):
    conn.execute('DELETE FROM gastos WHERE id=?', (gasto_id,))

def anular_ingreso(ingreso_id):
    conn.execute('DELETE FROM ingresos WHERE id=?', (ingreso_id,))
```

ALTO

S-7 -- anular_venta no reversa comision cuando el gasto ya fue sincronizado*Archivo: modules/database.py:1301*

Si el sync worker de Google Sheets ya proceso y marco el gasto de comision como sincronizado=1, la query de anulacion lo borrara de SQLite pero el registro ya existe en Google Sheets. Los reportes de Google Sheets mostraran comisiones que no existen en la base local.

```
conn.execute(
    'DELETE FROM gastos WHERE concepto LIKE ?',
    (f'Comision Tarjeta 4% {venta["folio"]}',)
)
```

ALTO

S-8 -- N+1 queries en listado de mesas (impacto en produccion)*Archivo: server.py:358-384*

Con 20 mesas y 3 ordenes cada una = 80+ queries por cada poll del frontend. Si el frontend hace polling cada 5 segundos, son ~1,000 queries/minuto solo para la vista de mesas.

```
for m in mesas_raw:                # query 1: todas las mesas
    ordenes = conn.execute(...)    # query por mesa
    for o in ordenes:
        items = conn.execute(...) # query por orden
```

MEDIO

S-9 -- Paginacion hardcodeda y sin offset*Archivo: modules/database.py:522, 541, 719*

No hay parametro offset ni cursor. Cuando el historial supere 200 registros, los mas antiguos son invisibles en la UI sin ningun aviso al usuario.

```
def listar_gastos(limit=200):      # sin offset
def listar_ingresos(limit=200):   # sin offset
def listar_nominas(limit=100):    # sin offset
```

MEDIO

S-10 -- Credencial de email en base de datos sin cifrar*Archivo: modules/database.py:1275-1281*

La contraseña del email se guarda en texto plano en config WHERE clave='email_password'. SQLite no tiene cifrado nativo, por lo que cualquier backup, volcado de base de datos o acceso fisico al archivo .db expone las credenciales de email.

Plan de Remediación Arquitectónica

Secuencia de implementación recomendada -- Bloques A -> B -> C -> D

Bloque A -- Seguridad crítica -- aplicar antes de continuar en producción

- 1 **Hashear credenciales del sistema:**
Mover SYSTEM_USERS a variables de entorno o tabla config, hashear passwords con bcrypt o argon2.
- 2 **Implementar sesiones/tokens en la API:**
Agregar FastAPI Depends con validación de token en todos los endpoints sensibles. El /api/login debe emitir un JWT o cookie firmada.
- 3 **Eliminar usuario_id del request body en operaciones privilegiadas:**
El usuario autenticado debe derivarse del token, no del body. Eliminar usuario_id de CerrarMesaReq, GastoReq, CorteReq, etc.
- 4 **Soft-delete para gastos e ingresos:**
Reemplazar DELETE FROM gastos/ingresos por UPDATE ... SET anulado=1, anulado_at=..., anulado_por=.... Agregar WHERE anulado IS NULL OR anulado=0 en todas las consultas de resumen.

Bloque B -- Integridad transaccional financiera

- 5 **Envolver cerrar_mesa y registrar_venta en transacciones atómicas:**
try/except con conn.rollback() en el bloque except y re-raise de la excepción.
- 6 **Corregir corregir_venta para manejar comisiones:**
Al cambiar método de pago, eliminar el gasto de comisión previo y, si el nuevo método es Tarjeta, crear uno nuevo con el monto correcto.
- 7 **Folio atómico con secuencia en config:**
Usar UPDATE config SET valor=valor+1 + SELECT en la misma transacción para garantizar unicidad sin condición de carrera.
- 8 **Proteger stock contra negativos:**
Agregar AND stock_local >= ? al UPDATE y verificar rowcount == 1; si es 0, abortar la transacción.

Bloque C -- Corrección de reportes financieros

- 9 **Unificar zona horaria:**
Elegir un solo origen de timestamps -- datetime('now') UTC en SQLite + conversión en Python, o datetime('now','localtime') consistente. No mezclar ambos.
- 10 **Corregir obtener_resumen_semana -- Bug #1:**
Alinear el filtro 'desde' (último corte) en el desglose diario del semanal para que sea consistente con obtener_resumen_dia.
- 11 **Corregir obtener_resumen_semana -- Bug #2:**
Restar gastos_caja del total_efectivo semanal con la misma lógica que el diario.
- 12 **Corregir obtener_resumen_semana -- Bug #3:**
Incluir inv (inversión/costo de mercancía) en la fórmula de utilidad semanal.
- 13 **Corregir obtener_resumen_semana -- Bug #4:**
Evitar doble-conteo de transferencias separando monto_tarjeta_puro y monto_transferencia en la consulta.

Bloque D -- Rendimiento y observabilidad

- 14 **Eliminar N+1 en api_mesas:**
Reemplazar el triple loop con dos queries (todas las órdenes abiertas + todos sus items) y agrupar en Python.
- 15 **Agregar paginación real:**
Parámetros offset: int = 0, limit: int = 50 en listar_gastos, listar_ingresos, listar_nominas. Retornar {"total": N, "items": [...]}.
[...]].
- 16 **Agregar tabla de auditoría:**
Crear audit_log (id, tabla, registro_id, acción, usuario_id, datos_anteriores, datos_nuevos, created_at) y escribir en cada anulación, corrección y eliminación.

Analisis de Discrepancias: Diario vs. Semanal

Causa raiz de los reportes financieros inconsistentes -- modules/database.py

CRITICO

Bug #1 -- El filtro 'desde' (corte) existe en el diario pero no en el semanal

Referencia: database.py:764-767 vs 954-958

El reporte diario filtra desde el ultimo corte de caja (turno actual). El desglose diario dentro del semanal filtra el dia completo sin considerar los cortes intermedios.

Efecto concreto: Si el lunes hubo dos turnos con corte a las 2pm -- Turno 1 (8am-2pm): \$5,000 / Turno 2 (2pm-10pm): \$3,000 -- el reporte diario del lunes mostrara \$3,000. El semanal mostrara \$8,000 para ese mismo lunes. Son datos distintos del mismo dia.

DIARIO (turno actual)

```
# obtener_resumen_dia
# Solo el turno actual
desde = last_corte['created_at']
WHERE DATE(created_at)=?
AND created_at > ?
```

SEMANAL (dia completo)

```
# obtener_resumen_semana
# Dia entero, sin filtro
# de turno/corte
WHERE DATE(created_at)
BETWEEN ? AND ?
```

ALTO

Bug #2 -- Formula del efectivo esperado es diferente entre diario y semanal

Referencia: database.py:854-858 vs 1103-1104

El diario resta los gastos de caja del total de efectivo para obtener el 'efectivo_esperado'. El semanal retorna el total de efectivo sin restar ningun gasto. Si en la semana hubo \$2,000 en gastos pagados en efectivo, el acumulado semanal de efectivo estara \$2,000 por encima de la suma de los diarios.

Diario (correcto)

```
# DIARIO -- resta gastos
efectivo_esperado =
total_ef - gastos_caja
```

Semanal (falta gastos_caja)

```
# SEMANAL -- sin restar
'total_efectivo':
rv['total_efectivo']
+ ingresos['ef']
```

ALTO

Bug #3 -- Formula de utilidad incompatible entre diario y semanal

Referencia: database.py:885 vs 1108

El diario descuenta el costo de los productos vendidos (inversion) de la utilidad. El semanal no. Si el costo de mercancia de la semana es \$10,000, la utilidad semanal aparecera \$10,000 mas alta que la suma de las utilidades diarias.

Diario (correcto)

```
# DIARIO -- incluye costo
utilidad =
total_ventas
- inv['inversion']
- total_gastos
```

Semanal (falta inversion)

```
# SEMANAL -- sin costo
utilidad =
total_semana
+ total_ingresos
- gastos['total']
```

MEDIO

Bug #4 -- Transferencias contadas dos veces en el semanal

Referencia: database.py:921-922 y 1112

Las transferencias se guardan en monto_tarjeta al momento de registrar la venta. El semanal expone tanto total_tarjeta (que incluye transferencias via monto_tarjeta) como total_transferencia (que las suma de nuevo por metodo_pago='Transferencia'). Ambos campos se muestran simultaneamente en la UI, inflando el total aparente.

Origen del dato

```
# En registrar_venta
monto_tarjeta = total
# Transfer guardada
# en monto_tarjeta
```

Doble conteo en semanal

```
# En resumen_semana
total_tarjeta =
    SUM(monto_tarjeta) # incl. transfer
total_transferencia =
    SUM(total) WHERE
    metodo='Transfer' # duplicado
```

Tabla resumen -- Causas raiz de discrepancias

Bug	Fuente de discrepancia	Sev.	Impacto en numeros
#1	Diario = turno actual / Semanal = dia completo	CRITICO	Diferencia = suma de turnos anteriores del dia
#2	Efectivo: semanal no resta gastos de caja	ALTO	Diferencia = total gastos caja de la semana
#3	Utilidad: semanal no resta costo de mercancia	ALTO	Diferencia = inversion/costo total de la semana
#4	Transferencias doble-contadas en semanal	MEDIO	Diferencia = total transferencias de la semana

Los Bugs #1-#4 estan localizados exclusivamente en obtener_resumen_semana

No requieren cambios en el esquema de base de datos. Son correcciones de logica en consultas SQL y calculos Python, correspondientes a los pasos 10-13 del Plan de Remediacion (Bloque C). Una vez aplicados, los totales diarios y el acumulado semanal deberan cuadrar exactamente.